Design Patterns & Code Quality

for Python Leads

A Complete Deep-Dive with Production-Grade Examples

Creational · Structural · Behavioral · Clean Architecture · DDD

Code Review Strategies · Refactoring Techniques

Table of Contents

PART 1: DESIGN PATTERNS

1. Creational Patterns

1.1 Singleton — Thread-safe, Decorator, __new__ approaches
1.2 Factory — Simple Factory & Abstract Factory
1.3 Builder — Fluent API with Director pattern

2. Structural Patterns

2.1 Adapter — Converting incompatible interfaces
2.2 Decorator — Dynamic behavior via wrapping
2.3 Facade — Simplifying complex subsystems

3. Behavioral Patterns

3.1 Observer — Event systems & pub/sub
3.2 Strategy — Interchangeable algorithms
3.3 Command — Encapsulated actions & undo/redo

PART 2: CODE QUALITY

4. Clean Architecture

5. Domain-Driven Design (DDD)

6. Code Review Strategies

7. Refactoring Techniques

Summary Cheat Sheet

PART 1: DESIGN PATTERNS

Purpose: Design patterns are reusable solutions to commonly occurring problems in software design. They are templates that can be applied to real-world programming situations. This section covers nine essential patterns grouped into Creational, Structural, and Behavioral categories.

1. CREATIONAL PATTERNS

PURPOSE

Control how objects are created, hiding complexity and promoting flexibility.

1.1 Singleton

INTENT

Guarantee a class has only one instance and provide a global access point.

WHEN TO USE

Database connections, loggers, configuration managers, thread pools.

```python
# ────────────────────────────────────────────────
# APPROACH 1: Thread-Safe Singleton (Metaclass)
# ────────────────────────────────────────────────
import threading
from typing import Any


class SingletonMeta(type):
    """
    A thread-safe Singleton metaclass.

    When a class uses this as its metaclass, only ONE instance
    of that class will ever exist.

    How it works:
    ─────────────
    1. `_instances` dict stores the single instance of each class.
    2. `_lock` ensures only one thread can create the instance.
    3. `__call__` is invoked whenever you do `MyClass()`.
       - First call  → creates the instance, stores it.
       - Later calls → returns the stored instance.
    """

    _instances: dict[type, Any] = {}
    _lock: threading.Lock = threading.Lock()

    def __call__(cls, *args, **kwargs):
        # Double-checked locking pattern:
        # First check (without lock) — fast path for subsequent calls
        if cls not in cls._instances:
            # Second check (with lock) — only one thread creates
            with cls._lock:
                if cls not in cls._instances:
                    instance = super().__call__(*args, **kwargs)
                    cls._instances[cls] = instance
        return cls._instances[cls]


class DatabaseConnection(metaclass=SingletonMeta):
    """Only one DB connection exists application-wide."""
```

```python
    def __init__(self, connection_string: str = "localhost:5432"):
        self.connection_string = connection_string
        self.connected = False
        print(f"[DatabaseConnection] Created with '{connection_string}'")

    def connect(self):
        self.connected = True
        print(f"[DatabaseConnection] Connected to '{self.connection_string}'")

    def execute(self, query: str) -> str:
        if not self.connected:
            raise RuntimeError("Not connected!")
        return f"Result of '{query}'"


# ────────────────────────────────────────────────
# APPROACH 2: Decorator-Based Singleton
# ────────────────────────────────────────────────
def singleton(cls):
    """A simpler decorator-based singleton (not thread-safe)."""
    instances = {}

    def get_instance(*args, **kwargs):
        if cls not in instances:
            instances[cls] = cls(*args, **kwargs)
        return instances[cls]

    return get_instance


@singleton
class AppConfig:
    def __init__(self):
        self.settings: dict[str, Any] = {}
        print("[AppConfig] Loaded configuration")

    def get(self, key: str, default: Any = None) -> Any:
        return self.settings.get(key, default)

    def set(self, key: str, value: Any) -> None:
        self.settings[key] = value


# ────────────────────────────────────────────────
# APPROACH 3: Module-Level Singleton (__new__)
# ────────────────────────────────────────────────
class Logger:
    """Singleton using __new__ — Pythonic approach."""

    _instance = None

    def __new__(cls, *args, **kwargs):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance._initialized = False
        return cls._instance

    def __init__(self, name: str = "app"):
        if self._initialized:
            return
        self.name = name
        self.logs: list[str] = []
        self._initialized = True
        print(f"[Logger] Initialized logger '{name}'")

    def log(self, message: str) -> None:
```

```python
        entry = f"[{self.name}] {message}"
        self.logs.append(entry)
        print(entry)


# ────────────────────────────────────────────────
# USAGE
# ────────────────────────────────────────────────
if __name__ == "__main__":
    # Metaclass Singleton
    db1 = DatabaseConnection("postgres://prod:5432")
    db2 = DatabaseConnection("postgres://other:5432")   # ← ignored!
    print(f"Same instance? {db1 is db2}")                # True
    print(f"Connection: {db2.connection_string}")         # prod:5432

    # Decorator Singleton
    config1 = AppConfig()
    config1.set("debug", True)
    config2 = AppConfig()
    print(f"Same? {config1 is config2}")                 # True
    print(f"debug = {config2.get('debug')}")             # True

    # __new__ Singleton
    log1 = Logger("myapp")
    log2 = Logger("other")
    print(f"Same? {log1 is log2}")                       # True
    log2.log("Hello!")                                   # [myapp] Hello!
```

```
[DatabaseConnection] Created with 'postgres://prod:5432'
Same instance? True
Connection: postgres://prod:5432
[AppConfig] Loaded configuration
Same? True
debug = True
[Logger] Initialized logger 'myapp'
Same? True
[myapp] Hello!
```

Key Considerations for a Lead:

- Singletons make unit testing harder (global state).
- Prefer dependency injection when possible.
- Use Singleton only for truly shared resources (config, logging, connection pools).

1.2 Factory

INTENT

Delegate object creation to a factory instead of calling constructors directly.

WHEN TO USE

When the exact class to instantiate depends on runtime data (config, user input, file type).

```python
# ────────────────────────────────────────────────
# SIMPLE FACTORY
# ────────────────────────────────────────────────
from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from enum import Enum, auto


# ── Product Interface ──
class Notification(ABC):
    """Abstract product — all notifications implement this."""

    @abstractmethod
    def send(self, recipient: str, message: str) -> str: ...

    @abstractmethod
    def validate(self, recipient: str) -> bool: ...


# ── Concrete Products ──
class EmailNotification(Notification):
    def send(self, recipient: str, message: str) -> str:
        if not self.validate(recipient):
            raise ValueError(f"Invalid email: {recipient}")
        return f"□ Email sent to {recipient}: {message}"

    def validate(self, recipient: str) -> bool:
        return "@" in recipient and "." in recipient


class SMSNotification(Notification):
    def send(self, recipient: str, message: str) -> str:
        if not self.validate(recipient):
            raise ValueError(f"Invalid phone: {recipient}")
        return f"□ SMS sent to {recipient}: {message}"

    def validate(self, recipient: str) -> bool:
        return recipient.startswith("+") and len(recipient) >= 10


class PushNotification(Notification):
    def send(self, recipient: str, message: str) -> str:
        return f"□ Push sent to device {recipient}: {message}"

    def validate(self, recipient: str) -> bool:
        return len(recipient) > 0


class SlackNotification(Notification):
    def send(self, recipient: str, message: str) -> str:
        return f"□ Slack sent to #{recipient}: {message}"

    def validate(self, recipient: str) -> bool:
        return not recipient.startswith("#")


# ── Simple Factory ──
class NotificationType(Enum):
    EMAIL = auto()
    SMS   = auto()
    PUSH  = auto()
    SLACK = auto()


class NotificationFactory:
```

```python
    """
    Simple Factory — one method decides which class to instantiate.

    Why use a factory instead of if/else in client code?
    ────────────────────────────────────────────────────
    1. Centralizes creation logic (one place to modify).
    2. Client code depends on the abstraction (Notification),
       not concrete classes.
    3. Easy to add new types without touching client code.
    """

    _registry: dict[NotificationType, type[Notification]] = {
        NotificationType.EMAIL: EmailNotification,
        NotificationType.SMS:   SMSNotification,
        NotificationType.PUSH:  PushNotification,
        NotificationType.SLACK: SlackNotification,
    }

    @classmethod
    def create(cls, notif_type: NotificationType) -> Notification:
        klass = cls._registry.get(notif_type)
        if klass is None:
            raise ValueError(f"Unknown notification type: {notif_type}")
        return klass()

    @classmethod
    def register(cls, notif_type: NotificationType,
                 klass: type[Notification]) -> None:
        """Open for extension — register new types at runtime."""
        cls._registry[notif_type] = klass


# ────────────────────────────────────────────────
# ABSTRACT FACTORY
# ────────────────────────────────────────────────
class UIComponent(ABC):
    @abstractmethod
    def render(self) -> str: ...

class Button(UIComponent): pass
class TextInput(UIComponent): pass
class Checkbox(UIComponent): pass

# ── Light Theme ──
class LightButton(Button):
    def render(self) -> str: return "[ Light Button ]"

class LightTextInput(TextInput):
    def render(self) -> str: return "| Light Input   |"

class LightCheckbox(Checkbox):
    def render(self) -> str: return "[✓] Light Check"

# ── Dark Theme ──
class DarkButton(Button):
    def render(self) -> str: return "[ Dark Button  ]"

class DarkTextInput(TextInput):
    def render(self) -> str: return "| Dark Input    |"

class DarkCheckbox(Checkbox):
    def render(self) -> str: return "[✓] Dark Check"


# ── Abstract Factory ──
class UIFactory(ABC):
```

```python
    """
    Abstract Factory creates FAMILIES of related objects.
    Each concrete factory produces a consistent set (theme).
    """
    @abstractmethod
    def create_button(self) -> Button: ...

    @abstractmethod
    def create_text_input(self) -> TextInput: ...

    @abstractmethod
    def create_checkbox(self) -> Checkbox: ...


class LightThemeFactory(UIFactory):
    def create_button(self)     -> Button:    return LightButton()
    def create_text_input(self) -> TextInput: return LightTextInput()
    def create_checkbox(self)   -> Checkbox:  return LightCheckbox()


class DarkThemeFactory(UIFactory):
    def create_button(self)     -> Button:    return DarkButton()
    def create_text_input(self) -> TextInput: return DarkTextInput()
    def create_checkbox(self)   -> Checkbox:  return DarkCheckbox()


def build_form(factory: UIFactory) -> list[str]:
    """Client code — works with ANY theme factory."""
    return [factory.create_button().render(),
            factory.create_text_input().render(),
            factory.create_checkbox().render()]


# ────────────────────────────────────────────────
# USAGE
# ────────────────────────────────────────────────
if __name__ == "__main__":
    # Simple Factory
    print("=== Simple Factory ===")
    email = NotificationFactory.create(NotificationType.EMAIL)
    sms   = NotificationFactory.create(NotificationType.SMS)
    print(email.send("alice@example.com", "Welcome!"))
    print(sms.send("+1234567890", "Your OTP is 5678"))

    # Abstract Factory
    print("\n=== Abstract Factory ===")
    for name, factory in [("Light", LightThemeFactory()),
                          ("Dark",  DarkThemeFactory())]:
        print(f"\n{name} Theme:")
        for component in build_form(factory):
            print(f"  {component}")
```

```
=== Simple Factory ===
 Email sent to alice@example.com: Welcome!
 SMS sent to +1234567890: Your OTP is 5678

=== Abstract Factory ===

Light Theme:
  [ Light Button ]
  | Light Input   |
  [✓] Light Check
```

```
Dark Theme:
  [ Dark Button  ]
  | Dark Input    |
  [✓] Dark Check
```

1.3 Builder

INTENT

Construct complex objects step by step, separating construction from representation.

WHEN TO USE

Objects with many optional parameters, complex configuration, building queries/documents/reports.

```python
from __future__ import annotations
from dataclasses import dataclass, field
from enum import Enum, auto
from typing import Optional


class Environment(Enum):
    DEV     = "development"
    STAGING = "staging"
    PROD    = "production"


@dataclass
class DatabaseConfig:
    host: str = "localhost"
    port: int = 5432
    name: str = "mydb"
    user: str = "admin"
    password: str = ""
    pool_size: int = 5
    ssl_enabled: bool = False


@dataclass
class CacheConfig:
    backend: str = "redis"
    host: str = "localhost"
    port: int = 6379
    ttl: int = 300


@dataclass
class ServerConfig:
    """
    Complex product with many interdependent settings.
    Creating this directly is error-prone — use a Builder.
    """
    app_name: str = "MyApp"
    environment: Environment = Environment.DEV
    host: str = "0.0.0.0"
    port: int = 8000
    debug: bool = True
```

```python
    workers: int = 1
    database: Optional[DatabaseConfig] = None
    cache: Optional[CacheConfig] = None
    allowed_hosts: list[str] = field(default_factory=list)
    middleware: list[str] = field(default_factory=list)
    cors_origins: list[str] = field(default_factory=list)
    log_level: str = "DEBUG"
    secret_key: str = "change-me"


# ────────────────────────────────────────────────
# THE BUILDER — step-by-step construction
# ────────────────────────────────────────────────
class ServerConfigBuilder:
    """
    Fluent Builder for ServerConfig.

    Why Builder instead of a big __init__?
    ──────────────────────────────────────
    1. Readable: `builder.with_database(...).with_cache(...)` reads like English.
    2. Validated: Each step can validate before setting.
    3. Preset configs: The Director provides common presets.
    4. Immutable result: Build once, use safely.
    """

    def __init__(self, app_name: str):
        self._config = ServerConfig(app_name=app_name)

    def set_environment(self, env: Environment) -> ServerConfigBuilder:
        self._config.environment = env
        return self

    def set_server(self, host: str = "0.0.0.0",
                   port: int = 8000) -> ServerConfigBuilder:
        self._config.host = host
        self._config.port = port
        return self

    def set_workers(self, count: int) -> ServerConfigBuilder:
        if count < 1:
            raise ValueError("Workers must be >= 1")
        self._config.workers = count
        return self

    def set_debug(self, enabled: bool) -> ServerConfigBuilder:
        self._config.debug = enabled
        return self

    def set_log_level(self, level: str) -> ServerConfigBuilder:
        valid = {"DEBUG", "INFO", "WARNING", "ERROR", "CRITICAL"}
        if level.upper() not in valid:
            raise ValueError(f"Invalid log level: {level}")
        self._config.log_level = level.upper()
        return self

    def with_database(self, host: str = "localhost", port: int = 5432,
                      name: str = "mydb", user: str = "admin",
                      password: str = "", pool_size: int = 5,
                      ssl: bool = False) -> ServerConfigBuilder:
        self._config.database = DatabaseConfig(
            host=host, port=port, name=name,
            user=user, password=password,
            pool_size=pool_size, ssl_enabled=ssl,
        )
        return self
```

```python
    def with_cache(self, backend: str = "redis",
                   host: str = "localhost", port: int = 6379,
                   ttl: int = 300) -> ServerConfigBuilder:
        self._config.cache = CacheConfig(
            backend=backend, host=host, port=port, ttl=ttl,
        )
        return self

    def add_middleware(self, *names: str) -> ServerConfigBuilder:
        self._config.middleware.extend(names)
        return self

    def set_allowed_hosts(self, *hosts: str) -> ServerConfigBuilder:
        self._config.allowed_hosts = list(hosts)
        return self

    def set_cors_origins(self, *origins: str) -> ServerConfigBuilder:
        self._config.cors_origins = list(origins)
        return self

    def set_secret_key(self, key: str) -> ServerConfigBuilder:
        if len(key) < 16:
            raise ValueError("Secret key must be >= 16 chars")
        self._config.secret_key = key
        return self

    def build(self) -> ServerConfig:
        """Validate and return the final product."""
        if self._config.environment == Environment.PROD:
            if self._config.debug:
                raise ValueError("Debug must be OFF in production!")
            if self._config.secret_key == "change-me":
                raise ValueError("Set a real secret key for production!")
            if not self._config.allowed_hosts:
                raise ValueError("Set allowed_hosts for production!")
        return self._config


# ────────────────────────────────────────────────
# THE DIRECTOR — provides preset configurations
# ────────────────────────────────────────────────
class ServerConfigDirector:
    """
    Director knows HOW to build common configurations.
    It uses the Builder's steps in a specific order.
    """

    @staticmethod
    def create_development(app_name: str) -> ServerConfig:
        return (
            ServerConfigBuilder(app_name)
            .set_environment(Environment.DEV)
            .set_debug(True)
            .set_log_level("DEBUG")
            .set_workers(1)
            .with_database(name=f"{app_name}_dev")
            .with_cache(ttl=60)
            .add_middleware("debug_toolbar", "cors")
            .build()
        )

    @staticmethod
    def create_production(app_name: str, secret_key: str,
                          db_host: str, domain: str) -> ServerConfig:
        return (
            ServerConfigBuilder(app_name)
```

```python
            .set_environment(Environment.PROD)
            .set_debug(False)
            .set_log_level("WARNING")
            .set_workers(8)
            .set_server("0.0.0.0", 443)
            .set_secret_key(secret_key)
            .set_allowed_hosts(domain, f"www.{domain}")
            .set_cors_origins(f"https://{domain}")
            .with_database(host=db_host, name=app_name,
                           pool_size=20, ssl=True)
            .with_cache(ttl=3600)
            .add_middleware("security_headers", "gzip",
                            "rate_limit", "cors")
            .build()
        )
```

```
╔═══════════════════════════════════╗
║                myapp              ║
╠═══════════════════════════════════╣
║  Env:      development            ║
║  Server:   0.0.0.0:8000           ║
║  Workers:  1                      ║
║  Debug:    True                   ║
║  Log:      DEBUG                  ║
║  DB:       localhost:5432/myapp_dev ║
║  Cache:    redis @ localhost:6379 ║
║  MW:       2 loaded               ║
╚═══════════════════════════════════╝

╔═══════════════════════════════════╗
║                myapp              ║
╠═══════════════════════════════════╣
║  Env:      production             ║
║  Server:   0.0.0.0:443            ║
║  Workers:  8                      ║
║  Debug:    False                  ║
║  Log:      WARNING                ║
║  DB:       db.prod.internal:5432/... ║
║  Cache:    redis @ localhost:6379 ║
║  MW:       4 loaded               ║
╚═══════════════════════════════════╝
```

2. STRUCTURAL PATTERNS

PURPOSE

Define how classes and objects are composed to form larger structures.

2.1 Adapter

INTENT

Convert the interface of a class into another interface clients expect. Makes incompatible interfaces work together.

WHEN TO USE

Integrating third-party libraries, migrating legacy code, unifying disparate APIs.

```python
from abc import ABC, abstractmethod
from dataclasses import dataclass
import xml.etree.ElementTree as ET


# ────────────────────────────────────────────────
# TARGET — the interface YOUR code expects
# ────────────────────────────────────────────────
@dataclass
class WeatherData:
    city: str
    temperature_celsius: float
    humidity_percent: float
    description: str


class WeatherService(ABC):
    """Interface that our application expects."""

    @abstractmethod
    def get_weather(self, city: str) -> WeatherData: ...


# ────────────────────────────────────────────────
# ADAPTEES — third-party services with DIFFERENT interfaces
# ────────────────────────────────────────────────
class OpenWeatherAPI:
    """Returns JSON with temperature in Fahrenheit."""

    def fetch_weather_data(self, city_name: str) -> dict:
        return {
            "city": city_name,
            "main": {"temp_f": 72.5, "humidity": 65},
            "weather": [{"description": "partly cloudy"}],
        }


class AccuWeatherAPI:
    """Returns XML with temperature in Kelvin."""
```

```python
    def get_conditions(self, location: str) -> str:
        return f"""
        <weather>
            <location>{location}</location>
            <temperature unit="kelvin">295.37</temperature>
            <relative_humidity>65</relative_humidity>
            <text_summary>Partly Cloudy</text_summary>
        </weather>
        """


class InternalLegacySystem:
    """Returns a tuple (city, temp_celsius, humidity, desc)."""

    def query(self, loc_id: str) -> tuple:
        return (loc_id, 22.5, 65.0, "Partly Cloudy")


# ────────────────────────────────────────────────
# ADAPTERS — bridge between adaptees and target
# ────────────────────────────────────────────────
class OpenWeatherAdapter(WeatherService):
    """
    Adapts OpenWeatherAPI → WeatherService.

    The adapter:
    1. Holds a reference to the adaptee.
    2. Implements the target interface.
    3. Translates calls & converts data.
    """

    def __init__(self, api: OpenWeatherAPI):
        self._api = api

    @staticmethod
    def _fahrenheit_to_celsius(f: float) -> float:
        return round((f - 32) * 5 / 9, 2)

    def get_weather(self, city: str) -> WeatherData:
        raw = self._api.fetch_weather_data(city)
        return WeatherData(
            city=raw["city"],
            temperature_celsius=self._fahrenheit_to_celsius(
                raw["main"]["temp_f"]
            ),
            humidity_percent=raw["main"]["humidity"],
            description=raw["weather"][0]["description"],
        )


class AccuWeatherAdapter(WeatherService):
    """Adapts AccuWeatherAPI (XML/Kelvin) → WeatherService."""

    def __init__(self, api: AccuWeatherAPI):
        self._api = api

    @staticmethod
    def _kelvin_to_celsius(k: float) -> float:
        return round(k - 273.15, 2)

    def get_weather(self, city: str) -> WeatherData:
        xml_str = self._api.get_conditions(city)
        root = ET.fromstring(xml_str.strip())
        temp_k = float(root.find("temperature").text)
        return WeatherData(
```

```python
            city=root.find("location").text,
            temperature_celsius=self._kelvin_to_celsius(temp_k),
            humidity_percent=float(root.find("relative_humidity").text),
            description=root.find("text_summary").text,
        )


class LegacySystemAdapter(WeatherService):
    """Adapts the legacy tuple-based system → WeatherService."""

    def __init__(self, system: InternalLegacySystem):
        self._system = system

    def get_weather(self, city: str) -> WeatherData:
        city_name, temp, humidity, desc = self._system.query(city)
        return WeatherData(city=city_name,
                           temperature_celsius=temp,
                           humidity_percent=humidity,
                           description=desc)


# ────────────────────────────────────────────────
# CLIENT CODE — works with ANY adapter
# ────────────────────────────────────────────────
def display_weather(service: WeatherService, city: str) -> None:
    """Client depends ONLY on WeatherService interface."""
    data = service.get_weather(city)
    print(f"  □ {data.city}")
    print(f"  □□  {data.temperature_celsius}°C")
    print(f"  □ {data.humidity_percent}%")
    print(f"  □ {data.description}")
    print()


if __name__ == "__main__":
    print("=== OpenWeather (Fahrenheit → Celsius) ===")
    display_weather(OpenWeatherAdapter(OpenWeatherAPI()), "London")

    print("=== AccuWeather (Kelvin XML → Celsius) ===")
    display_weather(AccuWeatherAdapter(AccuWeatherAPI()), "Paris")

    print("=== Legacy System (Tuple → Object) ===")
    display_weather(LegacySystemAdapter(InternalLegacySystem()), "NYC")
```

```
=== OpenWeather (Fahrenheit → Celsius) ===
  □ London
  □□  22.5°C
  □ 65%
  □ partly cloudy

=== AccuWeather (Kelvin XML → Celsius) ===
  □ Paris
  □□  22.22°C
  □ 65.0%
  □ Partly Cloudy

=== Legacy System (Tuple → Object) ===
  □ NYC
  □□  22.5°C
  □ 65.0%
  □ Partly Cloudy
```

2.2 Decorator

INTENT

Attach additional responsibilities to an object dynamically without modifying its code. Wrapping, not subclassing.

WHEN TO USE

Adding logging/caching/auth/retry/rate-limiting to existing services. Python's @decorator syntax is a language-level implementation of this pattern.

```python
from abc import ABC, abstractmethod
from functools import wraps
from typing import Any
import time


# ────────────────────────────────────────────────
# PATTERN APPROACH: Class-Based Decorators
# ────────────────────────────────────────────────

class DataSource(ABC):
    """Component interface."""
    @abstractmethod
    def read(self) -> str: ...

    @abstractmethod
    def write(self, data: str) -> None: ...


class FileDataSource(DataSource):
    def __init__(self, filename: str):
        self._filename = filename
        self._data = ""

    def read(self) -> str:
        print(f"    [File] Reading from '{self._filename}'")
        return self._data

    def write(self, data: str) -> None:
        print(f"    [File] Writing to '{self._filename}'")
        self._data = data


class DataSourceDecorator(DataSource, ABC):
    """
    Base decorator — wraps a DataSource.

    Key insight: The decorator IS-A DataSource AND HAS-A DataSource.
    This lets decorators be stacked: Encrypt(Compress(File(...)))
    """
    def __init__(self, source: DataSource):
        self._wrapped = source

    def read(self) -> str:
        return self._wrapped.read()

    def write(self, data: str) -> None:
```

```python
        self._wrapped.write(data)


class EncryptionDecorator(DataSourceDecorator):
    def __init__(self, source: DataSource, key: str = "secret"):
        super().__init__(source)
        self._key = key

    def _encrypt(self, data: str) -> str:
        key_bytes = (self._key * (len(data) // len(self._key) + 1)).encode()
        encrypted = bytes(a ^ b for a, b in zip(data.encode(), key_bytes))
        result = encrypted.hex()
        print(f"    [Encrypt] {data[:20]}... → {result[:20]}...")
        return result

    def _decrypt(self, data: str) -> str:
        encrypted = bytes.fromhex(data)
        key_bytes = (self._key * (len(encrypted) // len(self._key) + 1)).encode()
        decrypted = bytes(a ^ b for a, b in zip(encrypted, key_bytes)).decode()
        print(f"    [Decrypt] {data[:20]}... → {decrypted[:20]}...")
        return decrypted

    def write(self, data: str) -> None:
        super().write(self._encrypt(data))

    def read(self) -> str:
        return self._decrypt(super().read())


class CompressionDecorator(DataSourceDecorator):
    def _compress(self, data: str) -> str:
        compressed = f"COMPRESSED[{len(data)}]:{data}"
        print(f"    [Compress] {len(data)} chars → {len(compressed)} chars")
        return compressed

    def _decompress(self, data: str) -> str:
        if data.startswith("COMPRESSED["):
            idx = data.index("]:") + 2
            decompressed = data[idx:]
            print(f"    [Decompress] → {len(decompressed)} chars")
            return decompressed
        return data

    def write(self, data: str) -> None:
        super().write(self._compress(data))

    def read(self) -> str:
        return self._decompress(super().read())


class LoggingDecorator(DataSourceDecorator):
    def write(self, data: str) -> None:
        print(f"    [Log] WRITE operation, data length: {len(data)}")
        super().write(data)

    def read(self) -> str:
        print(f"    [Log] READ operation")
        result = super().read()
        print(f"    [Log] READ returned {len(result)} chars")
        return result


# ────────────────────────────────────────────────
# PYTHON-STYLE: Function Decorators
# ────────────────────────────────────────────────
```

```python
def retry(max_attempts: int = 3, delay: float = 0.1):
    """Decorator factory: retries a function on exception."""
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            last_exception = None
            for attempt in range(1, max_attempts + 1):
                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    last_exception = e
                    print(f"  ⚠  Attempt {attempt}/{max_attempts} failed: {e}")
                    if attempt < max_attempts:
                        time.sleep(delay)
            raise last_exception
        return wrapper
    return decorator


def cache(ttl_seconds: int = 60):
    """Decorator factory: caches function results."""
    _cache: dict[str, tuple[float, Any]] = {}

    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            key = f"{func.__name__}:{args}:{kwargs}"
            now = time.time()
            if key in _cache:
                cached_time, cached_value = _cache[key]
                if now - cached_time < ttl_seconds:
                    print(f"  ⚡ Cache HIT for {func.__name__}")
                    return cached_value
            print(f"  □ Cache MISS for {func.__name__}")
            result = func(*args, **kwargs)
            _cache[key] = (now, result)
            return result
        return wrapper
    return decorator


def timing(func):
    """Decorator: measures execution time."""
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        elapsed = time.perf_counter() - start
        print(f"  ⏱  {func.__name__} took {elapsed:.4f}s")
        return result
    return wrapper


call_count = 0

@timing
@retry(max_attempts=3, delay=0.05)
@cache(ttl_seconds=10)
def fetch_user_data(user_id: int) -> dict:
    """Simulates a flaky API call."""
    global call_count
    call_count += 1
    if call_count == 1:
        raise ConnectionError("API timeout")
    return {"id": user_id, "name": "Alice", "role": "admin"}
```

```python
# ────────────────────────────────────────────────
# USAGE — Stacking class-based decorators
# ────────────────────────────────────────────────
if __name__ == "__main__":
    # Stack: Logging → Encryption → Compression → File
    source = LoggingDecorator(
        EncryptionDecorator(
            CompressionDecorator(FileDataSource("data.txt")),
            key="mykey"
        )
    )

    print("--- Writing ---")
    source.write("Hello, Decorator Pattern!")

    print("--- Reading ---")
    result = source.read()
    print(f"Final result: '{result}'")

    print("--- Function Decorators ---")
    data = fetch_user_data(42)   # retries, then caches
    data = fetch_user_data(42)   # cache hit
```

```
--- Writing ---
    [Log] WRITE operation, data length: 25
    [Encrypt] Hello, Decorator Patter... → 200e07001a57170a1700...
    [Compress] 50 chars → 65 chars
    [File] Writing to 'data.txt'

--- Reading ---
    [Log] READ operation
    [File] Reading from 'data.txt'
    [Decompress] → 50 chars
    [Decrypt] 200e07001a57170a1700... → Hello, Decorator Patter...
    [Log] READ returned 25 chars
    Final result: 'Hello, Decorator Pattern!'

--- Function Decorators ---
  ⚠□  Attempt 1/3 failed: API timeout
  □ Cache MISS for fetch_user_data
  □□  fetch_user_data took 0.0523s
  ⚡ Cache HIT for fetch_user_data
  □□  fetch_user_data took 0.0001s
```

2.3 Facade

INTENT

Provide a simplified interface to a complex subsystem. Hide the wiring, expose the essentials.

WHEN TO USE

Simplifying complex library interactions, API gateways, service orchestration.

```python
from dataclasses import dataclass, field
from typing import Optional
import hashlib, time


# ────────────────────────────────────────────────
# COMPLEX SUBSYSTEM
# ────────────────────────────────────────────────
class InventorySystem:
    def __init__(self):
        self._stock: dict[str, int] = {
            "LAPTOP-001": 50, "PHONE-002": 200, "TABLET-003": 0,
        }

    def check_stock(self, product_id: str) -> int:
        stock = self._stock.get(product_id, 0)
        print(f"  [Inventory] {product_id}: {stock} in stock")
        return stock

    def reserve(self, product_id: str, quantity: int) -> bool:
        if self._stock.get(product_id, 0) >= quantity:
            self._stock[product_id] -= quantity
            print(f"  [Inventory] Reserved {quantity}x {product_id}")
            return True
        print(f"  [Inventory] □ Cannot reserve {quantity}x {product_id}")
        return False

    def release(self, product_id: str, quantity: int) -> None:
        self._stock[product_id] = self._stock.get(product_id, 0) + quantity


class PricingEngine:
    _prices = {"LAPTOP-001": 999.99, "PHONE-002": 699.99, "TABLET-003": 449.99}
    _tax_rates = {"US-CA": 0.0725, "US-NY": 0.08, "UK": 0.20}

    def get_price(self, product_id: str) -> float:
        price = self._prices.get(product_id, 0)
        print(f"  [Pricing] {product_id} base price: ${price}")
        return price

    def apply_discount(self, price: float, code: str) -> float:
        discounts = {"SAVE10": 0.10, "SAVE20": 0.20, "VIP": 0.15}
        discount = discounts.get(code, 0)
        final = round(price * (1 - discount), 2)
        print(f"  [Pricing] Discount '{code}': ${price} → ${final}")
        return final

    def calculate_tax(self, price: float, region: str) -> float:
        rate = self._tax_rates.get(region, 0)
        tax = round(price * rate, 2)
        print(f"  [Pricing] Tax for {region}: ${tax}")
        return tax


class PaymentGateway:
    def authorize(self, card_token: str, amount: float) -> Optional[str]:
        if amount > 5000:
            print(f"  [Payment] □ Amount ${amount} exceeds limit")
            return None
        txn_id = hashlib.md5(f"{card_token}{amount}".encode()).hexdigest()[:12]
        print(f"  [Payment] □ Authorized ${amount}, txn: {txn_id}")
        return txn_id

    def capture(self, txn_id: str) -> bool:
        print(f"  [Payment] □ Captured txn {txn_id}")
        return True
```

```python
    def refund(self, txn_id: str, amount: float) -> bool:
        print(f"  [Payment] ↩  Refunded ${amount} for txn {txn_id}")
        return True


class ShippingService:
    _rates = {"standard": 5.99, "express": 15.99, "overnight": 29.99}

    def get_rate(self, method: str, weight_kg: float) -> float:
        base = self._rates.get(method, 5.99)
        rate = round(base + (weight_kg * 0.5), 2)
        print(f"  [Shipping] {method}: ${rate} ({weight_kg}kg)")
        return rate

    def create_shipment(self, address: str, method: str) -> str:
        tracking = f"TRK-{hashlib.md5(address.encode()).hexdigest()[:8]}"
        print(f"  [Shipping] □ Created shipment {tracking}")
        return tracking


class NotificationService:
    def send_email(self, to: str, subject: str, body: str) -> None:
        print(f"  [Notify] □ Email to {to}: {subject}")


# ────────────────────────────────────────────────
# THE FACADE — simple interface to all subsystems
# ────────────────────────────────────────────────
@dataclass
class OrderResult:
    success: bool
    order_id: str = ""
    total: float = 0.0
    tracking: str = ""
    error: str = ""


class OrderFacade:
    """
    Facade: ONE simple method to place an order.

    Without this facade, the client would need to:
    1. Know about 5 different subsystem classes
    2. Call them in the correct order
    3. Handle rollbacks if any step fails
    4. Manage the data flow between subsystems
    """

    def __init__(self):
        self._inventory  = InventorySystem()
        self._pricing    = PricingEngine()
        self._payment    = PaymentGateway()
        self._shipping   = ShippingService()
        self._notifier   = NotificationService()

    def place_order(self, product_id: str, quantity: int,
                    card_token: str, shipping_address: str, email: str,
                    region: str = "US-CA", discount_code: str = "",
                    shipping_method: str = "standard") -> OrderResult:
        order_id = f"ORD-{int(time.time())}"

        # Step 1: Inventory
        stock = self._inventory.check_stock(product_id)
        if stock < quantity:
            return OrderResult(success=False,
```

```python
                                error=f"Insufficient stock: {stock} < {quantity}")
        if not self._inventory.reserve(product_id, quantity):
            return OrderResult(success=False, error="Reservation failed")

        # Step 2: Pricing
        unit_price = self._pricing.get_price(product_id)
        subtotal   = unit_price * quantity
        if discount_code:
            subtotal = self._pricing.apply_discount(subtotal, discount_code)
        tax      = self._pricing.calculate_tax(subtotal, region)
        shipping = self._shipping.get_rate(shipping_method, quantity * 0.5)
        total    = round(subtotal + tax + shipping, 2)

        # Step 3: Payment
        txn_id = self._payment.authorize(card_token, total)
        if not txn_id:
            self._inventory.release(product_id, quantity)
            return OrderResult(success=False, error="Payment declined")
        self._payment.capture(txn_id)

        # Step 4: Shipping
        tracking = self._shipping.create_shipment(shipping_address, shipping_method)

        # Step 5: Notification
        self._notifier.send_email(email, f"Order {order_id} Confirmed!", f"Total: ${total}")

        return OrderResult(success=True, order_id=order_id,
                           total=total, tracking=tracking)


# ────────────────────────────────────────────────
# USAGE — Client code is beautifully simple
# ────────────────────────────────────────────────
if __name__ == "__main__":
    shop = OrderFacade()

    result = shop.place_order(
        product_id="LAPTOP-001", quantity=2,
        card_token="tok_visa_4242",
        shipping_address="123 Main St, San Francisco, CA",
        email="alice@example.com",
        discount_code="SAVE10",
        shipping_method="express",
    )
    print(f"\n Order: {result.order_id}, Total: ${result.total}")

    # Failed order (out of stock)
    result = shop.place_order(
        product_id="TABLET-003", quantity=1,
        card_token="tok_visa_4242",
        shipping_address="456 Oak Ave",
        email="bob@example.com",
    )
    print(f"\n Error: {result.error}")
```

3. BEHAVIORAL PATTERNS

PURPOSE

Define how objects communicate and divide responsibilities.

3.1 Observer

INTENT

Define a one-to-many dependency so that when one object changes state, all dependents are notified automatically.

WHEN TO USE

Event systems, UI updates, pub/sub, reactive programming.

```python
from __future__ import annotations
from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from datetime import datetime
from typing import Any, Callable


# ────────────────────────────────────────────────
# APPROACH 1: Classic OOP Observer
# ────────────────────────────────────────────────
@dataclass
class Event:
    """Data carried with each notification."""
    name: str
    data: dict[str, Any]
    timestamp: datetime = field(default_factory=datetime.now)


class Observer(ABC):
    @abstractmethod
    def update(self, event: Event) -> None: ...


class Subject:
    """
    Observable subject — maintains a list of observers.

    Key concepts:
    • Observers subscribe to specific event types.
    • When state changes, relevant observers are notified.
    • Subject doesn't know concrete observer types (loose coupling).
    """
    def __init__(self):
        self._observers: dict[str, list[Observer]] = {}

    def subscribe(self, event_name: str, observer: Observer) -> None:
        if event_name not in self._observers:
            self._observers[event_name] = []
        self._observers[event_name].append(observer)
```

```python
    def unsubscribe(self, event_name: str, observer: Observer) -> None:
        if event_name in self._observers:
            self._observers[event_name].remove(observer)

    def notify(self, event: Event) -> None:
        for observer in self._observers.get(event.name, []):
            observer.update(event)


class ECommerceStore(Subject):
    def new_order(self, order_id: str, customer: str, total: float):
        print(f"\n□ New order: {order_id} by {customer}, ${total}")
        self.notify(Event("order_placed", {
            "order_id": order_id, "customer": customer, "total": total,
        }))

    def cancel_order(self, order_id: str, reason: str):
        print(f"\n□ Order cancelled: {order_id}")
        self.notify(Event("order_cancelled", {
            "order_id": order_id, "reason": reason,
        }))

    def item_back_in_stock(self, product_id: str, product_name: str):
        print(f"\n□ Back in stock: {product_name}")
        self.notify(Event("back_in_stock", {
            "product_id": product_id, "product_name": product_name,
        }))


class EmailNotifier(Observer):
    def update(self, event: Event) -> None:
        if event.name == "order_placed":
            print(f"    □ [Email] Confirmation to {event.data['customer']}")
        elif event.name == "order_cancelled":
            print(f"    □ [Email] Cancellation notice sent")
        elif event.name == "back_in_stock":
            print(f"    □ [Email] '{event.data['product_name']}' available!")


class InventoryManager(Observer):
    def update(self, event: Event) -> None:
        if event.name == "order_placed":
            print(f"    □ [Inventory] Reserving for {event.data['order_id']}")
        elif event.name == "order_cancelled":
            print(f"    □ [Inventory] Releasing for {event.data['order_id']}")


class AnalyticsTracker(Observer):
    def __init__(self):
        self.events_tracked: list[str] = []

    def update(self, event: Event) -> None:
        self.events_tracked.append(event.name)
        print(f"    □ [Analytics] '{event.name}' "
              f"(total: {len(self.events_tracked)})")


class FraudDetector(Observer):
    def update(self, event: Event) -> None:
        if event.name == "order_placed":
            total = event.data.get("total", 0)
            if total > 1000:
                print(f"    □ [Fraud] HIGH VALUE ${total} — flagged!")
            else:
                print(f"    □ [Fraud] Order ${total} — looks normal")
```

```python
# ────────────────────────────────────────────────
# APPROACH 2: Pythonic Event System (functions)
# ────────────────────────────────────────────────
class EventBus:
    """A more Pythonic observer using callable handlers."""

    def __init__(self):
        self._handlers: dict[str, list[Callable]] = {}

    def on(self, event_name: str, handler: Callable) -> Callable:
        if event_name not in self._handlers:
            self._handlers[event_name] = []
        self._handlers[event_name].append(handler)
        return handler

    def emit(self, event_name: str, **kwargs) -> None:
        for handler in self._handlers.get(event_name, []):
            handler(**kwargs)


# ────────────────────────────────────────────────
# USAGE
# ────────────────────────────────────────────────
if __name__ == "__main__":
    store = ECommerceStore()
    email = EmailNotifier(); inventory = InventoryManager()
    analytics = AnalyticsTracker(); fraud = FraudDetector()

    store.subscribe("order_placed",   email)
    store.subscribe("order_placed",   inventory)
    store.subscribe("order_placed",   analytics)
    store.subscribe("order_placed",   fraud)
    store.subscribe("order_cancelled", email)
    store.subscribe("order_cancelled", inventory)
    store.subscribe("back_in_stock",   email)

    store.new_order("ORD-001", "alice@example.com", 299.99)
    store.new_order("ORD-002", "bob@example.com",  2499.99)
    store.cancel_order("ORD-001", "Customer changed mind")
    store.item_back_in_stock("TAB-003", "iPad Pro")

    # Pythonic EventBus
    bus = EventBus()

    @bus.on("user_signup")
    def send_welcome(username: str, email: str, **_):
        print(f"  □ Welcome email to {email}")

    @bus.on("user_signup")
    def create_profile(username: str, **_):
        print(f"  □ Created profile for {username}")

    bus.emit("user_signup", username="charlie", email="charlie@test.com")
```

3.2 Strategy

INTENT

Define a family of algorithms, encapsulate each one, and make them interchangeable at runtime.

WHEN TO USE

Multiple ways to sort, validate, authenticate, price, compress, or export data.

```python
from abc import ABC, abstractmethod
from dataclasses import dataclass
from typing import Callable


# ────────────────────────────────────────────────
# APPROACH 1: Classic OOP Strategy
# ────────────────────────────────────────────────
@dataclass
class ShippingOrder:
    weight_kg: float
    distance_km: float
    is_fragile: bool = False
    is_express: bool = False


class ShippingStrategy(ABC):
    """Each strategy encapsulates a different pricing algorithm."""

    @abstractmethod
    def calculate_cost(self, order: ShippingOrder) -> float: ...

    @abstractmethod
    def estimated_days(self, order: ShippingOrder) -> int: ...

    @property
    @abstractmethod
    def name(self) -> str: ...


class StandardShipping(ShippingStrategy):
    @property
    def name(self) -> str: return "Standard Ground"

    def calculate_cost(self, order: ShippingOrder) -> float:
        return round(5.99 + order.weight_kg * 0.50
                     + order.distance_km * 0.01
                     + (3.00 if order.is_fragile else 0), 2)

    def estimated_days(self, order: ShippingOrder) -> int:
        return max(3, int(order.distance_km / 500))


class ExpressShipping(ShippingStrategy):
    @property
    def name(self) -> str: return "Express Air"

    def calculate_cost(self, order: ShippingOrder) -> float:
        return round(15.99 + order.weight_kg * 1.50
                     + order.distance_km * 0.03
                     + (8.00 if order.is_fragile else 0), 2)

    def estimated_days(self, order: ShippingOrder) -> int:
        return 1 if order.distance_km < 1000 else 2


class FreeShipping(ShippingStrategy):
    @property
    def name(self) -> str: return "Free (Promo)"
```

```python
    def calculate_cost(self, order: ShippingOrder) -> float: return 0.0

    def estimated_days(self, order: ShippingOrder) -> int:
        return max(5, int(order.distance_km / 300))


class PickupStrategy(ShippingStrategy):
    @property
    def name(self) -> str: return "Store Pickup"

    def calculate_cost(self, order: ShippingOrder) -> float: return 0.0
    def estimated_days(self, order: ShippingOrder) -> int: return 0


class ShippingCalculator:
    """
    Context that uses a strategy.

    The calculator doesn't know HOW shipping is calculated.
    It delegates to whatever strategy is set.
    Strategies can be swapped at runtime.
    """
    def __init__(self, strategy: ShippingStrategy):
        self._strategy = strategy

    @property
    def strategy(self) -> ShippingStrategy:
        return self._strategy

    @strategy.setter
    def strategy(self, strategy: ShippingStrategy) -> None:
        print(f"  □ Strategy changed to: {strategy.name}")
        self._strategy = strategy

    def calculate(self, order: ShippingOrder) -> dict:
        return {
            "method": self._strategy.name,
            "cost":   self._strategy.calculate_cost(order),
            "days":   self._strategy.estimated_days(order),
        }


# ────────────────────────────────────────────────
# APPROACH 2: Pythonic Strategy (functions)
# ────────────────────────────────────────────────
@dataclass
class TextDocument:
    content: str
    title: str = "Untitled"


def export_as_markdown(doc: TextDocument) -> str:
    return f"# {doc.title}\n\n{doc.content}"

def export_as_html(doc: TextDocument) -> str:
    return f"<html><body><h1>{doc.title}</h1><p>{doc.content}</p></body></html>"

def export_as_plain(doc: TextDocument) -> str:
    return f"{doc.title}\n{'=' * len(doc.title)}\n{doc.content}"

def export_as_json(doc: TextDocument) -> str:
    import json
    return json.dumps({"title": doc.title, "content": doc.content}, indent=2)
```

```python
ExportStrategy = Callable[[TextDocument], str]


class DocumentExporter:
    """Context using function-based strategies."""
    _strategies: dict[str, ExportStrategy] = {
        "markdown": export_as_markdown,
        "html":     export_as_html,
        "plain":    export_as_plain,
        "json":     export_as_json,
    }

    def __init__(self, default_format: str = "plain"):
        self._format = default_format

    @classmethod
    def register_format(cls, name: str, strategy: ExportStrategy) -> None:
        cls._strategies[name] = strategy

    def export(self, doc: TextDocument, fmt: str | None = None) -> str:
        format_name = fmt or self._format
        strategy = self._strategies.get(format_name)
        if not strategy:
            raise ValueError(f"Unknown format: {format_name}")
        return strategy(doc)


if __name__ == "__main__":
    order = ShippingOrder(weight_kg=2.5, distance_km=800, is_fragile=True)
    calc  = ShippingCalculator(StandardShipping())

    for strategy in [StandardShipping(), ExpressShipping(),
                     FreeShipping(), PickupStrategy()]:
        calc.strategy = strategy
        result = calc.calculate(order)
        print(f"    {result['method']:>16}: ${result['cost']:>7.2f} "
              f"| {result['days']} days")

    doc = TextDocument("Strategy pattern is powerful!", "Design Patterns")
    exporter = DocumentExporter()
    for fmt in ["plain", "markdown", "html", "json"]:
        print(f"\n--- {fmt.upper()} ---")
        print(exporter.export(doc, fmt))
```

3.3 Command

INTENT

Encapsulate a request as an object, allowing you to parameterize clients, queue operations, and support undo/redo.

WHEN TO USE

Undo/redo, task queues, macro recording, transaction logs.

```python
from __future__ import annotations
from abc import ABC, abstractmethod
```

```python
from dataclasses import dataclass, field


class Command(ABC):
    """
    Each command encapsulates an action and its inverse.

    Key properties:
    • execute()     — performs the action
    • undo()        — reverses the action
    • description   — human-readable name
    """
    @abstractmethod
    def execute(self) -> None: ...

    @abstractmethod
    def undo(self) -> None: ...

    @property
    @abstractmethod
    def description(self) -> str: ...


class TextDocument:
    """Receiver: a simple text editor document."""

    def __init__(self):
        self._lines: list[str] = []

    @property
    def text(self) -> str: return "\n".join(self._lines)

    @property
    def line_count(self) -> int: return len(self._lines)

    def insert_line(self, index: int, text: str) -> None:
        self._lines.insert(index, text)

    def remove_line(self, index: int) -> str:
        return self._lines.pop(index)

    def get_line(self, index: int) -> str:
        return self._lines[index]

    def replace_line(self, index: int, text: str) -> str:
        old = self._lines[index]
        self._lines[index] = text
        return old

    def display(self) -> None:
        print("    ┌──────────────────────────────────┐")
        if not self._lines:
            print("    │  (empty document)                │")
        for i, line in enumerate(self._lines):
            print(f"    │ {i+1:2}│ {line:<30}│")
        print("    └──────────────────────────────────┘")


class InsertLineCommand(Command):
    def __init__(self, document: TextDocument, index: int, text: str):
        self._doc, self._index, self._text = document, index, text

    @property
    def description(self) -> str:
        return f"Insert '{self._text}' at line {self._index + 1}"
```

```python
    def execute(self) -> None: self._doc.insert_line(self._index, self._text)
    def undo(self)    -> None: self._doc.remove_line(self._index)


class DeleteLineCommand(Command):
    def __init__(self, document: TextDocument, index: int):
        self._doc, self._index = document, index
        self._deleted_text: str = ""

    @property
    def description(self) -> str: return f"Delete line {self._index + 1}"

    def execute(self) -> None:
        self._deleted_text = self._doc.remove_line(self._index)

    def undo(self) -> None:
        self._doc.insert_line(self._index, self._deleted_text)


class ReplaceLineCommand(Command):
    def __init__(self, document: TextDocument, index: int, new_text: str):
        self._doc, self._index, self._new_text = document, index, new_text
        self._old_text: str = ""

    @property
    def description(self) -> str:
        return f"Replace line {self._index + 1} with '{self._new_text}'"

    def execute(self) -> None:
        self._old_text = self._doc.replace_line(self._index, self._new_text)

    def undo(self) -> None:
        self._doc.replace_line(self._index, self._old_text)


class MacroCommand(Command):
    """A composite command — executes multiple commands as one."""

    def __init__(self, name: str, commands: list[Command]):
        self._name, self._commands = name, commands

    @property
    def description(self) -> str:
        return f"Macro '{self._name}' ({len(self._commands)} commands)"

    def execute(self) -> None:
        for cmd in self._commands: cmd.execute()

    def undo(self) -> None:
        for cmd in reversed(self._commands): cmd.undo()


class TextEditor:
    """
    Invoker: executes commands and manages undo/redo stacks.

    This is the core of the Command pattern's power:
    • Every action is recorded
    • Full undo/redo support
    • Action history / audit trail
    """
    def __init__(self):
        self.document = TextDocument()
        self._history: list[Command] = []
        self._redo_stack: list[Command] = []
```

```python
    def execute(self, command: Command) -> None:
        command.execute()
        self._history.append(command)
        self._redo_stack.clear()
        print(f"  □ {command.description}")

    def undo(self) -> None:
        if not self._history:
            print("  ⚠□  Nothing to undo"); return
        command = self._history.pop()
        command.undo()
        self._redo_stack.append(command)
        print(f"  ↩□  Undo: {command.description}")

    def redo(self) -> None:
        if not self._redo_stack:
            print("  ⚠□  Nothing to redo"); return
        command = self._redo_stack.pop()
        command.execute()
        self._history.append(command)
        print(f"  ↪□  Redo: {command.description}")

    def show_history(self) -> None:
        print("\n  □ Command History:")
        for i, cmd in enumerate(self._history, 1):
            print(f"     {i}. {cmd.description}")
```

```
□ Insert 'Hello, World!' at line 1
□ Insert 'This is the Command pattern.' at line 2
□ Insert 'Each action is an object.' at line 3

  ┌──────────────────────────────┐
  │  1│ Hello, World!            │
  │  2│ This is the Command pattern.│
  │  3│ Each action is an object.  │
  └──────────────────────────────┘

□ Replace line 1 with 'Hi, Design Patterns!'
↩□  Undo: Replace line 1 with 'Hi, Design Patterns!'
↩□  Undo: Insert 'Each action is an object.' at line 3
↪□  Redo: Insert 'Each action is an object.' at line 3
□ Macro 'add_signature' (3 commands)

  ┌──────────────────────────────┐
  │  1│ Hello, World!            │
  │  2│ This is the Command pattern.│
  │  3│ Each action is an object.  │
  │  4│ ---                      │
  │  5│ Author: Python Lead      │
  │  6│ Date: 2024               │
  └──────────────────────────────┘

↩□  Undo: Macro 'add_signature' (3 commands)
```

PART 2: CODE QUALITY

Code quality is not just about making code that works — it's about writing code that is readable, maintainable, testable, and secure. As a Python Lead, you're responsible for setting the standard. This section covers the architecture, design philosophy, review process, and refactoring techniques that separate good engineers from great ones.

4. Clean Architecture

CORE PRINCIPLE

Dependency flows INWARD. Business logic never depends on frameworks, databases, or UI. The inner layers know nothing about the outer layers.

Architectural Layers

```
┌─────────────────────────────────────────────────────────────┐
│                                                             │
│   ┌─────────────────────────────────────────────────────┐   │
│   │                                                     │   │
│   │   ┌─────────────────────────────────────────────┐   │   │
│   │   │                                             │   │   │
│   │   │   ┌─────────────────────────────┐           │   │   │
│   │   │   │       ENTITIES              │           │   │   │
│   │   │   │    (Business Objects)       │           │   │   │
│   │   │   └─────────────────────────────┘           │   │   │
│   │   │                                             │   │   │
│   │   │      USE CASES                              │   │   │
│   │   │   (Application Business Rules)              │   │   │
│   │   └─────────────────────────────────────────────┘   │   │
│   │                                                     │   │
│   │      INTERFACE ADAPTERS                             │   │
│   │   (Controllers, Gateways, Presenters)               │   │
│   └─────────────────────────────────────────────────────┘   │
│                                                             │
│      FRAMEWORKS & DRIVERS                                   │
│   (Web, DB, External APIs, UI)                              │
│                                                             │
└─────────────────────────────────────────────────────────────┘

          ↑ Dependencies point INWARD ↑
```

Layer Responsibilities:

- Entities — Core business objects. Zero dependencies. Pure Python dataclasses.
- Use Cases — Application business rules. Orchestrate entities. Depend only on domain interfaces.
- Interface Adapters — Controllers, presenters, repository implementations. Convert between layers.
- Frameworks & Drivers — Web frameworks, databases, external APIs. The outermost ring.

```
"""
Clean Architecture Example: User Registration System

Directory structure:
────────────────────

project/
├── domain/              ← INNERMOST (no dependencies)
│   ├── entities.py
│   └── repositories.py  (interfaces only)
├── use_cases/           ← depends only on domain
```

```
│   └── register_user.py
├── adapters/            ← depends on domain + use_cases
│   ├── controllers.py
│   ├── presenters.py
│   └── repositories.py  (implementations)
└── frameworks/          ← OUTERMOST
    ├── web.py
    └── database.py
"""

# ═══════════════════════════════════════════════
# LAYER 1: DOMAIN (innermost — zero dependencies)
# ═══════════════════════════════════════════════
from __future__ import annotations
from dataclasses import dataclass, field
from datetime import datetime
from abc import ABC, abstractmethod
from typing import Optional
import re, uuid


@dataclass
class User:
    """
    Domain Entity — pure business object.
    Contains ONLY business logic and validation.
    No database code, no framework imports.
    """
    id: str = field(default_factory=lambda: str(uuid.uuid4()))
    username: str = ""
    email: str = ""
    password_hash: str = ""
    is_active: bool = True
    created_at: datetime = field(default_factory=datetime.utcnow)

    def __post_init__(self):
        self._validate()

    def _validate(self) -> None:
        if self.username and len(self.username) < 3:
            raise ValueError("Username must be >= 3 characters")
        if self.email and not self._is_valid_email(self.email):
            raise ValueError(f"Invalid email: {self.email}")

    @staticmethod
    def _is_valid_email(email: str) -> bool:
        return bool(re.match(
            r"^[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-.]+$", email
        ))


@dataclass
class RegistrationResult:
    success: bool
    user: Optional[User] = None
    error: str = ""


class UserRepository(ABC):
    """
    Repository INTERFACE — defined in domain layer.
    The domain says WHAT it needs (find, save),
    but NOT HOW (SQL, Mongo, file).
    """
    @abstractmethod
    def find_by_email(self, email: str) -> Optional[User]: ...
```

```python
    @abstractmethod
    def find_by_username(self, username: str) -> Optional[User]: ...
    @abstractmethod
    def save(self, user: User) -> None: ...


class PasswordHasher(ABC):
    @abstractmethod
    def hash(self, password: str) -> str: ...
    @abstractmethod
    def verify(self, password: str, hashed: str) -> bool: ...


class EventPublisher(ABC):
    @abstractmethod
    def publish(self, event_name: str, data: dict) -> None: ...


# ═══════════════════════════════════════════════════
# LAYER 2: USE CASES (depends only on domain)
# ═══════════════════════════════════════════════════
@dataclass
class RegisterUserRequest:
    username: str
    email: str
    password: str


class RegisterUserUseCase:
    """
    Application Business Rule.
    Orchestrates: Validate → Check uniqueness →
                  Create entity → Save → Publish event

    Depends on INTERFACES (abstractions), not concrete
    implementations. → Dependency Inversion Principle.
    """
    def __init__(self, user_repo: UserRepository,
                 password_hasher: PasswordHasher,
                 event_publisher: EventPublisher):
        self._repo    = user_repo
        self._hasher  = password_hasher
        self._events  = event_publisher

    def execute(self, request: RegisterUserRequest) -> RegistrationResult:
        if len(request.password) < 8:
            return RegistrationResult(
                success=False, error="Password must be >= 8 characters")

        if self._repo.find_by_email(request.email):
            return RegistrationResult(
                success=False,
                error=f"Email '{request.email}' already registered")

        if self._repo.find_by_username(request.username):
            return RegistrationResult(
                success=False,
                error=f"Username '{request.username}' already taken")

        try:
            user = User(
                username=request.username,
                email=request.email,
                password_hash=self._hasher.hash(request.password),
            )
        except ValueError as e:
```

```python
            return RegistrationResult(success=False, error=str(e))

        self._repo.save(user)
        self._events.publish("user_registered", {
            "user_id": user.id, "username": user.username,
            "email": user.email,
        })
        return RegistrationResult(success=True, user=user)


# ═══════════════════════════════════════════════════
# LAYER 3: ADAPTERS (implements domain interfaces)
# ═══════════════════════════════════════════════════
class InMemoryUserRepository(UserRepository):
    def __init__(self):
        self._users: dict[str, User] = {}

    def find_by_email(self, email: str) -> Optional[User]:
        return next((u for u in self._users.values()
                     if u.email == email), None)

    def find_by_username(self, username: str) -> Optional[User]:
        return next((u for u in self._users.values()
                     if u.username == username), None)

    def save(self, user: User) -> None:
        self._users[user.id] = user
        print(f"    [Repo] Saved user: {user.username} ({user.id[:8]}...)")


class SimplePasswordHasher(PasswordHasher):
    def hash(self, password: str) -> str:
        import hashlib
        return hashlib.sha256(password.encode()).hexdigest()

    def verify(self, password: str, hashed: str) -> bool:
        return self.hash(password) == hashed


class ConsoleEventPublisher(EventPublisher):
    def publish(self, event_name: str, data: dict) -> None:
        print(f"    [Event] {event_name}: {data}")


class RegistrationController:
    """Translates HTTP-like input to use case input and back."""
    def __init__(self, use_case: RegisterUserUseCase):
        self._use_case = use_case

    def handle(self, request_data: dict) -> dict:
        uc_request = RegisterUserRequest(
            username=request_data.get("username", ""),
            email=request_data.get("email", ""),
            password=request_data.get("password", ""),
        )
        result = self._use_case.execute(uc_request)
        if result.success:
            return {"status": 201, "body": {
                "message": "User created successfully",
                "user_id": result.user.id,
                "username": result.user.username,
            }}
        return {"status": 400, "body": {"error": result.error}}


# ═══════════════════════════════════════════════════
```

```python
# LAYER 4: FRAMEWORKS (outermost — wiring)
# ═══════════════════════════════════════════════
def create_app():
    """
    Composition Root — wires everything together.
    The ONLY place that knows about ALL concrete classes.
    Swap implementations without touching business logic.
    """
    controller = RegistrationController(
        RegisterUserUseCase(
            InMemoryUserRepository(),
            SimplePasswordHasher(),
            ConsoleEventPublisher(),
        )
    )
    return controller


if __name__ == "__main__":
    controller = create_app()

    response = controller.handle({
        "username": "alice",
        "email": "alice@example.com",
        "password": "securepass123",
    })
    print(f"Response: {response}")

    # Duplicate email
    response = controller.handle({
        "username": "bob",
        "email": "alice@example.com",
        "password": "anotherpass",
    })
    print(f"Response: {response}")
```

BENEFITS OF CLEAN ARCHITECTURE

□ Business logic has ZERO framework dependencies | □ Easy to test (mock the interfaces) | □ Easy to swap (change DB: only adapters change) | □ Framework-agnostic (Django → FastAPI? Only outer layer changes)

5. Domain-Driven Design (DDD)

CORE IDEA

Model software around the business domain, using a ubiquitous language shared between developers and domain experts.

Key DDD Building Blocks

- Entity — Has identity and lifecycle (Order, Customer). Equality by identity.
- Value Object — Defined by attributes, immutable (Money, Address). Equality by value.

- Aggregate — Cluster of entities with a root (Order → OrderLines). Enforces invariants.
- Repository — Persistence abstraction defined in the domain layer.
- Domain Event — Something that happened in the domain (OrderPlaced, OrderShipped).
- Domain Service — Logic that doesn't belong to a single entity (PricingService).
- Specification — Encapsulated, composable business rule (HighValueOrder & MultiItemOrder).

```python
"""
DDD Example: Order Management System

Key DDD Concepts Demonstrated:
──────────────────────────────
1. Entity       — has identity, lifecycle (Order, Customer)
2. Value Object — defined by attributes, immutable (Money, Address)
3. Aggregate    — cluster of entities with a root (Order → OrderLines)
4. Repository   — persistence abstraction
5. Domain Event — something that happened in the domain
6. Domain Service — logic that doesn't belong to a single entity
7. Specification — encapsulated business rule
"""
from __future__ import annotations
from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from datetime import datetime
from decimal import Decimal
from enum import Enum, auto
from typing import Optional
import uuid


# ═══════════════════════════════════════════════
# VALUE OBJECTS — immutable, compared by value
# ═══════════════════════════════════════════════
@dataclass(frozen=True)
class Money:
    """
    Value Object: Amount + Currency.
    Immutable. Two Money objects are equal if
    amount AND currency are equal.
    """
    amount: Decimal
    currency: str = "USD"

    def __post_init__(self):
        if self.amount < 0:
            raise ValueError("Money amount cannot be negative")

    def __add__(self, other: Money) -> Money:
        if self.currency != other.currency:
            raise ValueError(f"Cannot add {self.currency} and {other.currency}")
        return Money(self.amount + other.amount, self.currency)

    def __mul__(self, quantity: int) -> Money:
        return Money(self.amount * quantity, self.currency)

    def __gt__(self, other: Money) -> bool:
        return self.amount > other.amount

    def __ge__(self, other: Money) -> bool:
        return self.amount >= other.amount

    def __str__(self) -> str:
```

```python
        return f"{self.currency} {self.amount:.2f}"


@dataclass(frozen=True)
class Address:
    street: str
    city: str
    state: str
    zip_code: str
    country: str = "US"

    def __str__(self) -> str:
        return f"{self.street}, {self.city}, {self.state} {self.zip_code}"


@dataclass(frozen=True)
class ProductId:
    value: str
    def __str__(self) -> str: return self.value


# ═══════════════════════════════════════════════
# DOMAIN EVENTS
# ═══════════════════════════════════════════════
@dataclass(frozen=True)
class DomainEvent:
    occurred_at: datetime = field(default_factory=datetime.utcnow)


@dataclass(frozen=True)
class OrderPlaced(DomainEvent):
    order_id: str = ""
    customer_id: str = ""
    total: Money = field(default_factory=lambda: Money(Decimal("0")))


@dataclass(frozen=True)
class OrderShipped(DomainEvent):
    order_id: str = ""
    tracking_number: str = ""


# ═══════════════════════════════════════════════
# AGGREGATE ROOT — Order
# ═══════════════════════════════════════════════
class OrderStatus(Enum):
    DRAFT = auto(); PLACED = auto(); PAID = auto()
    SHIPPED = auto(); DELIVERED = auto(); CANCELLED = auto()


@dataclass
class OrderLine:
    id: str = field(default_factory=lambda: str(uuid.uuid4())[:8])
    product_id: ProductId = field(default_factory=lambda: ProductId(""))
    product_name: str = ""
    unit_price: Money = field(default_factory=lambda: Money(Decimal("0")))
    quantity: int = 1

    @property
    def line_total(self) -> Money:
        return self.unit_price * self.quantity


class Order:
    """
    AGGREGATE ROOT — the main entity.
```

```python
    Rules:
    • ALL modifications go through the aggregate root.
    • External code never directly modifies OrderLine.
    • The root enforces all invariants (business rules).
    • Domain events are collected here.
    """
    MAX_LINES = 20
    MIN_ORDER_TOTAL = Money(Decimal("10.00"))

    def __init__(self, customer_id: str, shipping_address: Address):
        self.id = str(uuid.uuid4())[:12]
        self.customer_id = customer_id
        self.shipping_address = shipping_address
        self.status = OrderStatus.DRAFT
        self.created_at = datetime.utcnow()
        self._lines: list[OrderLine] = []
        self._events: list[DomainEvent] = []

    @property
    def lines(self) -> tuple[OrderLine, ...]:
        return tuple(self._lines)

    @property
    def total(self) -> Money:
        if not self._lines: return Money(Decimal("0"))
        result = self._lines[0].line_total
        for line in self._lines[1:]:
            result = result + line.line_total
        return result

    def add_line(self, product_id: ProductId, product_name: str,
                 unit_price: Money, quantity: int = 1) -> OrderLine:
        self._ensure_status(OrderStatus.DRAFT)
        if len(self._lines) >= self.MAX_LINES:
            raise ValueError(f"Max {self.MAX_LINES} lines exceeded")
        if quantity < 1:
            raise ValueError("Quantity must be >= 1")
        line = OrderLine(product_id=product_id, product_name=product_name,
                         unit_price=unit_price, quantity=quantity)
        self._lines.append(line)
        return line

    def place(self) -> None:
        self._ensure_status(OrderStatus.DRAFT)
        if not self._lines:
            raise ValueError("Cannot place an empty order")
        if not (self.total >= self.MIN_ORDER_TOTAL):
            raise ValueError(f"Minimum order: {self.MIN_ORDER_TOTAL}")
        self.status = OrderStatus.PLACED
        self._events.append(OrderPlaced(
            order_id=self.id, customer_id=self.customer_id,
            total=self.total,
        ))

    def mark_paid(self) -> None:
        self._ensure_status(OrderStatus.PLACED)
        self.status = OrderStatus.PAID

    def ship(self, tracking_number: str) -> None:
        self._ensure_status(OrderStatus.PAID)
        self.status = OrderStatus.SHIPPED
        self._events.append(OrderShipped(
            order_id=self.id, tracking_number=tracking_number,
        ))
```

```python
    def cancel(self, reason: str) -> None:
        if self.status in (OrderStatus.SHIPPED, OrderStatus.DELIVERED):
            raise ValueError("Cannot cancel a shipped/delivered order")
        self.status = OrderStatus.CANCELLED

    def _ensure_status(self, expected: OrderStatus) -> None:
        if self.status != expected:
            raise ValueError(
                f"Order must be {expected.name}, but is {self.status.name}")


# ═══════════════════════════════════════════════
# SPECIFICATION PATTERN
# ═══════════════════════════════════════════════
class Specification(ABC):
    @abstractmethod
    def is_satisfied_by(self, candidate) -> bool: ...

    def __and__(self, other): return AndSpecification(self, other)
    def __or__(self, other):  return OrSpecification(self, other)
    def __invert__(self):     return NotSpecification(self)


class AndSpecification(Specification):
    def __init__(self, left, right): self._l, self._r = left, right
    def is_satisfied_by(self, c): return (self._l.is_satisfied_by(c) and
                                          self._r.is_satisfied_by(c))


class OrSpecification(Specification):
    def __init__(self, left, right): self._l, self._r = left, right
    def is_satisfied_by(self, c): return (self._l.is_satisfied_by(c) or
                                          self._r.is_satisfied_by(c))


class NotSpecification(Specification):
    def __init__(self, spec): self._s = spec
    def is_satisfied_by(self, c): return not self._s.is_satisfied_by(c)


class HighValueOrder(Specification):
    def __init__(self, threshold=Money(Decimal("500"))):
        self._threshold = threshold
    def is_satisfied_by(self, order: Order) -> bool:
        return order.total >= self._threshold


class MultiItemOrder(Specification):
    def __init__(self, min_lines: int = 3): self._min = min_lines
    def is_satisfied_by(self, order: Order) -> bool:
        return len(order.lines) >= self._min


# ═══════════════════════════════════════════════
# DOMAIN SERVICE
# ═══════════════════════════════════════════════
class PricingService:
    """
    Domain Service: calculates discounts.
    This logic doesn't belong to Order or Customer alone.
    """
    @staticmethod
    def calculate_discount(order: Order) -> Money:
        total = order.total
        if len(order.lines) >= 3:
            rate = Decimal("0.05")
```

```
        elif total >= Money(Decimal("500")):
            rate = Decimal("0.10")
        else:
            rate = Decimal("0")
        return Money((total.amount * rate).quantize(Decimal("0.01")))
```

6. Code Review Strategies

PHILOSOPHY

Code reviews are not about finding bugs — they're about sharing knowledge, enforcing standards, and improving the codebase together. As a lead, set the tone: be constructive, specific, and educational.

What to Review

□ Correctness

- Logic is correct, edge cases handled
- Boundary conditions considered
- Concurrent access considered (thread safety)

□ Security

- No SQL/command injection vulnerabilities
- Secrets not hardcoded in source
- Input validated and sanitized
- Proper authentication/authorization checks

□ Design (SOLID)

- Single Responsibility: each class has one reason to change
- Open/Closed: open for extension, closed for modification
- Liskov Substitution: subtypes are substitutable
- Interface Segregation: no fat interfaces
- Dependency Inversion: depend on abstractions

□ Readability

- Clear naming (variables, functions, classes)
- Functions are small and focused (<20 lines ideal)
- Comments explain WHY, not WHAT
- Type hints on all public interfaces

□ Performance

- No obvious O(n²) when O(n) exists
- Database queries are efficient (N+1 query problem?)
- Async used where appropriate

□ Testing

- Tests exist for new/changed code
- Tests cover happy path + error cases
- Tests are independent and deterministic
- Mocks used appropriately (not excessively)

```python
# ═══════════════════════════════════════════════
# WHAT TO LOOK FOR IN CODE REVIEWS
# ═══════════════════════════════════════════════

# ─────────────────────────────────────────────
# 1. CORRECTNESS — Does it actually work?
# ─────────────────────────────────────────────

# □ BAD: Off-by-one, race condition, missing edge cases
def find_max_bad(numbers):
    max_val = 0  # Bug: fails for negative numbers
    for n in numbers:
        if n > max_val:
            max_val = n
    return max_val

# □ GOOD: Handles edge cases
def find_max_good(numbers: list[float]) -> float:
    if not numbers:
        raise ValueError("Cannot find max of empty list")
    return max(numbers)  # Use stdlib!


# ─────────────────────────────────────────────
# 2. SECURITY — Is it safe?
# ─────────────────────────────────────────────

# □ BAD: SQL injection vulnerability
def get_user_bad(username: str):
    query = f"SELECT * FROM users WHERE name = '{username}'"
    # If username = "'; DROP TABLE users; --" ... □
    return query

# □ GOOD: Parameterized queries
def get_user_good(cursor, username: str):
    cursor.execute("SELECT * FROM users WHERE name = %s", (username,))
    return cursor.fetchone()


# ─────────────────────────────────────────────
# 3. DESIGN — SOLID Principles
# ─────────────────────────────────────────────

# □ BAD: God class violating Single Responsibility
class UserManagerBad:
    def create_user(self, data): ...
    def send_email(self, user, msg): ...
    def generate_pdf_report(self, users): ...
    def authenticate(self, token): ...
```

```python
    def update_database(self, query): ...
    def validate_credit_card(self, card): ...

# □ GOOD: Each class has ONE reason to change
class UserService:
    def create_user(self, data): ...
    def get_user(self, user_id): ...

class EmailService:
    def send(self, to, subject, body): ...

class ReportGenerator:
    def generate(self, data, format): ...


# ────────────────────────────────────────────
# 4. ERROR HANDLING
# ────────────────────────────────────────────

# □ BAD: Swallowing exceptions
def process_bad(data):
    try:
        result = complex_operation(data)
    except Exception:
        pass  # Silently fails — bugs disappear
    return None

# □ GOOD: Specific exceptions, proper handling
def process_good(data: dict) -> float:
    try:
        value = int(data["value"])
    except KeyError:
        raise ValueError("Missing 'value' key in data")
    except (TypeError, ValueError) as e:
        raise ValueError(f"'value' must be an integer: {e}")

    try:
        return value / data["divisor"]
    except ZeroDivisionError:
        raise ValueError("Divisor cannot be zero")
    except KeyError:
        raise ValueError("Missing 'divisor' key in data")


# ────────────────────────────────────────────
# 5. PERFORMANCE — Obvious inefficiencies
# ────────────────────────────────────────────

# □ BAD: O(n²) when O(n) exists
def find_duplicates_bad(items: list) -> list:
    duplicates = []
    for i, item in enumerate(items):
        if item in items[i+1:]:  # O(n) search each time!
            duplicates.append(item)
    return duplicates

# □ GOOD: O(n) with a set
def find_duplicates_good(items: list) -> set:
    seen, duplicates = set(), set()
    for item in items:
        if item in seen: duplicates.add(item)
        seen.add(item)
    return duplicates


# ────────────────────────────────────────────
```

```python
# 6. TESTABILITY — Is it easy to test?
# ────────────────────────────────────────────────

# ❌ BAD: Hard-coded dependency, impossible to mock
import datetime as dt

class ReportBad:
    def generate(self):
        now  = dt.datetime.now()  # Can't control in tests!
        data = open("report.csv").read()  # Hard-coded file!

# ✅ GOOD: Dependencies injected
class ReportGood:
    def __init__(self, data_source, clock=None):
        self._source = data_source
        self._clock  = clock or dt.datetime

    def generate(self):
        now  = self._clock.now()     # Easily mocked
        data = self._source.read()   # Easily mocked
```

7. Refactoring Techniques

KEY PRINCIPLE

Refactoring means improving code without changing its observable behavior. Always have tests BEFORE refactoring. Make ONE change at a time. Run tests after EACH change.

1. Extract Method

Break long functions into small, focused functions with clear names. Each function should do ONE thing.

2. Replace Conditionals with Polymorphism

Replace if/elif type-checking chains with abstract classes and subclasses. Let each class know its own behavior.

3. Replace Magic Numbers with Named Constants

Replace bare literals with named constants or Enum values for clarity and maintainability.

4. Introduce Parameter Object

When a function takes 5+ related parameters, group them into a dataclass.

5. Guard Clauses (Fail Fast)

Replace deep nesting with early returns. Validate preconditions at the top and return early. The happy path should have no nesting.

6. Context Managers for Resources

Replace manual try/finally with the 'with' statement. Use @contextmanager for custom lifecycle.

7. Composition over Inheritance

Replace deep class hierarchies with Protocol-based composition. Objects are composed of capabilities, not defined by ancestry.

8. Introduce Explaining Variable

Replace complex expressions with named variables that document the intent.

```python
# ═══════════════════════════════════════════════════
# TECHNIQUE 1: Extract Method
# Purpose: Break long functions into readable pieces
# ═══════════════════════════════════════════════════
from dataclasses import dataclass
from typing import Optional


@dataclass
class OrderItem:
    name: str
    price: float
    quantity: int


@dataclass
class OrderSummary:
    subtotal: float
    discount: float
    tax: float
    total: float


def validate_order(order_data: dict) -> Optional[str]:
    """Return error message or None if valid."""
    if not order_data.get("customer_id"):
        return "Missing customer ID"
    if not order_data.get("items"):
        return "No items"
    for item in order_data["items"]:
        if item.get("quantity", 0) <= 0:
            return f"Invalid quantity for {item['name']}"
        if item.get("price", 0) <= 0:
            return f"Invalid price for {item['name']}"
    return None


def calculate_subtotal(items: list[OrderItem]) -> float:
    return sum(item.price * item.quantity for item in items)


def calculate_discount(subtotal: float) -> float:
    if subtotal > 100: return subtotal * 0.10
    elif subtotal > 50: return subtotal * 0.05
    return 0.0
```

```python
def calculate_tax(amount: float, rate: float = 0.08) -> float:
    return round(amount * rate, 2)


def process_order(order_data: dict) -> OrderSummary | dict:
    error = validate_order(order_data)
    if error:
        return {"error": error}
    items    = [OrderItem(**item) for item in order_data["items"]]
    subtotal = calculate_subtotal(items)
    discount = calculate_discount(subtotal)
    taxable  = subtotal - discount
    tax      = calculate_tax(taxable)
    return OrderSummary(subtotal=subtotal, discount=discount,
                        tax=tax, total=round(taxable + tax, 2))


# ═══════════════════════════════════════════════════
# TECHNIQUE 2: Replace Conditionals with Polymorphism
# ═══════════════════════════════════════════════════
from abc import ABC, abstractmethod
import math


class Shape(ABC):
    @abstractmethod
    def area(self) -> float: ...

    @abstractmethod
    def perimeter(self) -> float: ...

    def describe(self) -> str:
        return (f"{self.__class__.__name__}: "
                f"area={self.area():.2f}, "
                f"perimeter={self.perimeter():.2f}")


@dataclass
class Circle(Shape):
    radius: float
    def area(self)      -> float: return math.pi * self.radius ** 2
    def perimeter(self) -> float: return 2 * math.pi * self.radius


@dataclass
class Rectangle(Shape):
    width: float; height: float
    def area(self)      -> float: return self.width * self.height
    def perimeter(self) -> float: return 2 * (self.width + self.height)


@dataclass
class Triangle(Shape):
    base: float; height: float; a: float; b: float; c: float
    def area(self)      -> float: return 0.5 * self.base * self.height
    def perimeter(self) -> float: return self.a + self.b + self.c


# ═══════════════════════════════════════════════════
# TECHNIQUE 3: Replace Magic Numbers/Strings
# ═══════════════════════════════════════════════════
from enum import Enum


class ShippingMethod(Enum):
    STANDARD  = "standard"
```

```python
    EXPRESS   = "express"
    OVERNIGHT = "overnight"


@dataclass(frozen=True)
class ShippingRate:
    per_kg: float
    base_fee: float

    def calculate(self, weight_kg: float) -> float:
        return round(weight_kg * self.per_kg + self.base_fee, 2)


SHIPPING_RATES: dict[ShippingMethod, ShippingRate] = {
    ShippingMethod.STANDARD:  ShippingRate(per_kg=0.50, base_fee=5.99),
    ShippingMethod.EXPRESS:   ShippingRate(per_kg=1.20, base_fee=12.99),
    ShippingMethod.OVERNIGHT: ShippingRate(per_kg=2.00, base_fee=24.99),
}


def calculate_shipping(weight_kg: float, method: ShippingMethod) -> float:
    rate = SHIPPING_RATES.get(method)
    if not rate: raise ValueError(f"Unknown method: {method}")
    return rate.calculate(weight_kg)


# ═══════════════════════════════════════════════════
# TECHNIQUE 4: Guard Clauses (Replace Nesting)
# ═══════════════════════════════════════════════════

# □ BEFORE: Deep nesting (arrow anti-pattern)
def process_payment_bad(payment):
    if payment is not None:
        if payment.amount > 0:
            if payment.currency in ("USD", "EUR", "GBP"):
                if payment.card_valid:
                    if payment.fraud_check_passed:
                        return "charged"   # 5 levels deep!
                    else: return "Fraud detected"
                else: return "Invalid card"
            else: return "Unsupported currency"
        else: return "Invalid amount"
    else: return "No payment"


# □ AFTER: Guard clauses — fail fast, flat code
def process_payment(payment) -> str:
    if payment is None:                              return "No payment"
    if payment.amount <= 0:                          return "Invalid amount"
    if payment.currency not in ("USD", "EUR", "GBP"): return "Unsupported currency"
    if not payment.card_valid:                       return "Invalid card"
    if not payment.fraud_check_passed:               return "Fraud detected"
    return "charged"   # Happy path — no nesting!


# ═══════════════════════════════════════════════════
# TECHNIQUE 5: Composition over Inheritance
# ═══════════════════════════════════════════════════
from typing import Protocol


class CanFly(Protocol):
    def fly(self) -> str: ...

class CanSwim(Protocol):
    def swim(self) -> str: ...
```

```python
@dataclass
class Wings:
    span_meters: float
    def fly(self) -> str: return f"Flying with {self.span_meters}m wingspan"


@dataclass
class Fins:
    count: int
    def swim(self) -> str: return f"Swimming with {self.count} fins"


@dataclass
class Duck:
    """Composed of capabilities, not inherited."""
    name: str
    wings: Wings = field(default_factory=lambda: Wings(0.3))
    fins: Fins = field(default_factory=lambda: Fins(2))

    def fly(self)  -> str: return self.wings.fly()
    def swim(self) -> str: return self.fins.swim()


# ────────────────────────────────────────────────
# DEMO
# ────────────────────────────────────────────────
if __name__ == "__main__":
    result = process_order({
        "customer_id": "C001",
        "items": [
            {"name": "Widget", "price": 29.99, "quantity": 3},
            {"name": "Gadget", "price": 49.99, "quantity": 1},
        ],
    })
    print(f"Order: {result}")

    shapes: list[Shape] = [Circle(5), Rectangle(4, 6), Triangle(3, 4, 3, 4, 5)]
    for shape in shapes:
        print(shape.describe())

    for method in ShippingMethod:
        print(f"  {method.value:>10}: ${calculate_shipping(2.5, method)}")

    duck = Duck("Donald")
    print(f"{duck.name}: {duck.fly()}")
    print(f"{duck.name}: {duck.swim()}")
```

```
Order: OrderSummary(subtotal=139.96, discount=13.996, tax=10.08, total=136.04)
Circle: area=78.54, perimeter=31.42
Rectangle: area=24.00, perimeter=20.00
Triangle: area=6.00, perimeter=12.00
    standard: $7.24
     express: $15.99
   overnight: $29.99
Donald: Flying with 0.3m wingspan
Donald: Swimming with 2 fins
```

Summary: Quick Reference Cheat Sheet

```
╔═══════════════════════════════════════════════════════════╗
║           DESIGN PATTERNS & CODE QUALITY                  ║
║                Quick Reference Card                       ║
╚═══════════════════════════════════════════════════════════╝
║                                                           ║
║  CREATIONAL                                               ║
║  Singleton  → One instance only (DB, Config, Logger)      ║
║  Factory    → Delegate creation (if/else → polymorphism)  ║
║  Builder    → Complex objects step-by-step (fluent API)   ║
║                                                           ║
║  STRUCTURAL                                               ║
║  Adapter    → Convert interface A → interface B           ║
║  Decorator  → Wrap objects to add behavior dynamically    ║
║  Facade     → Simple API for complex subsystems           ║
║                                                           ║
║  BEHAVIORAL                                               ║
║  Observer   → Event system: subject notifies observers    ║
║  Strategy   → Swap algorithms at runtime                  ║
║  Command    → Encapsulate actions (undo/redo, queues)     ║
║                                                           ║
║  CODE QUALITY                                             ║
║  Clean Arch → Dependencies flow inward (domain is pure)   ║
║  DDD        → Model around business domain concepts       ║
║  Reviews    → Correctness → Security → Design → Tests     ║
║  Refactor   → Extract, Guard, Compose, Polymorphism       ║
║                                                           ║
║  PYTHON-SPECIFIC TIPS                                     ║
║  • Use Protocol over ABC when possible                     ║
║  • Prefer function strategies over class strategies        ║
║  • dataclass + frozen=True for value objects               ║
║  • @contextmanager for resource lifecycle                  ║
║  • Type hints on all public interfaces                     ║
║  • Dependency injection > global singletons               ║
║                                                           ║
╚═══════════════════════════════════════════════════════════╝
```

Python-Specific Best Practices

- **Use Protocol over ABC when possible**

  Protocols enable structural subtyping (duck typing with type safety) — more Pythonic than inheritance.

- **Prefer function strategies over class strategies**

  Python functions are first-class objects. A Callable is often enough instead of a full Strategy class.

- **dataclass + frozen=True for value objects**

  Immutable dataclasses with __hash__ enabled are perfect for DDD value objects.

- **@contextmanager for resource lifecycle**

  Cleaner than implementing __enter__/__exit__. Use for file handles, DB transactions, timing.

- **Type hints on all public interfaces**

  Enables IDE support, mypy checking, and serves as executable documentation.

- **Dependency injection over global singletons**
  Pass dependencies through constructors/parameters. Makes code testable and decoupled.
- **f-strings over .format() or %**
  More readable, faster at runtime. Use for all string interpolation in Python 3.6+.
- **Pathlib over os.path**
  Object-oriented, cross-platform, more readable. Path('/tmp/file') vs os.path.join('/tmp', 'file').

This guide covers the fundamental design patterns and code quality practices essential for Python technical leads. The patterns presented here are not rules to follow blindly — they are tools to reach for when the situation calls for them. The best code is code that your team can understand, modify, and test with confidence.